

Programación

SIMULACRO PRÁCTICO – RECUPERACIÓN TEMAS 1-7

(11/06/2026)



LEE ATENTAMENTE LAS SIGUIENTES INSTRUCCIONES ANTES DE EMPEZAR:



- Si estás leyendo esto es porque ya te has descargado el enunciado del examen. Así que, desde este momento **ya puedes desconectar el cable Ethernet de tu equipo.**
- Cuando termines de resolver el examen, **avisa a tu profe** para conectar de nuevo el cable Ethernet y realizar la entrega.
- **Sube el proyecto Java comprimido (.zip) a la entrega de AULES disponible.**

PARTE 1: Configuración del entorno

1. Crea un nuevo proyecto Java (Maven) con IntelliJ -o el IDE que utilices-. Llámalo “SIMULACRO EXTRAORDINARIA”.
2. Crea en el proyecto dos paquetea nuevos llamados “culpa_padel” y “visita_papa” para ir añadiendo las clases correspondientes al problema planteado.

PARTE 2: Resolución de problemas

Programa en Java la solución al siguiente problema. Usa el proyecto que te acabas de crear en el apartado anterior.

(2,5p) PROBLEMA 1: POR QUÉ PIERDO AL PÁDEL

Desde que eres alumno del instituto IES MUTXAMEL, justo en frente del club *URBAN PADEL MTX*, has encontrado en el pádel una forma perfecta de relacionarte más con los compis de tu clase y mantenerte en forma. Sin embargo, después de cada partido perdido (que, siendo sinceros, son muchos), te queda una sensación extraña. Algo dentro de ti te dice que la culpa no es sólo tuya...

Después de un profundo análisis, has identificado los principales factores que afectan a tus resultados:

- El 54% de la culpa es de tu **compañero o compañera**. No importa quién sea, siempre falla en los momentos clave, deja pasar bolas fáciles o simplemente no cubre su parte de la pista.

- El 12% se debe a la **pista**. A veces hay viento, otras veces el bote no es el esperado, o incluso hay pequeñas irregularidades en el suelo que sólo parecen afectarte a ti.

- El 25% es por culpa de la **pala**. O pesa demasiado, o tiene poco agarre, o su balance no es el adecuado. Da igual cuál uses, siempre le pasa algo.

- El 8% es por la dichosa **pelota**. Hay unas que botan una barbaridad, otras no botan nada. Así no hay quién se aclare.

Y ya está, así que el 1% de las veces tienes que reconocer que es **culpa tuya**. A veces no das lo mejor de ti mismo, es verdad.

ENTRADA:

La entrada del programa consta de una línea con 4 números en formato **N-N-N-N**: el porcentaje de culpa que se atribuye al compañero/a, a la pista, a la pala y a la pelota. Todos ellos son números enteros entre 1 y 90, y la suma de todos ellos siempre debe ser menor que 100:

```
*** CULPA PÁDEL APP ***
Introduce el porcentaje de culpa de los 4 factores externos (compi-pista-pala-pelota):
52-12-25-8
```

Si la suma es mayor a 100, el programa debe volver a pedir los datos:

```
*** CULPA PÁDEL APP ***
Introduce el porcentaje de culpa de los 4 factores externos (compi-pista-pala-pelota):
52-12-25-8

*** CULPA PÁDEL APP ***
Introduce el porcentaje de culpa de los 4 factores externos (compi-pista-pala-pelota):
1-1-1-100
ERROR. La suma es mayor a 100.

Introduce el porcentaje de culpa de los 4 factores externos (compi-pista-pala-pelota):
|
```

SALIDA:

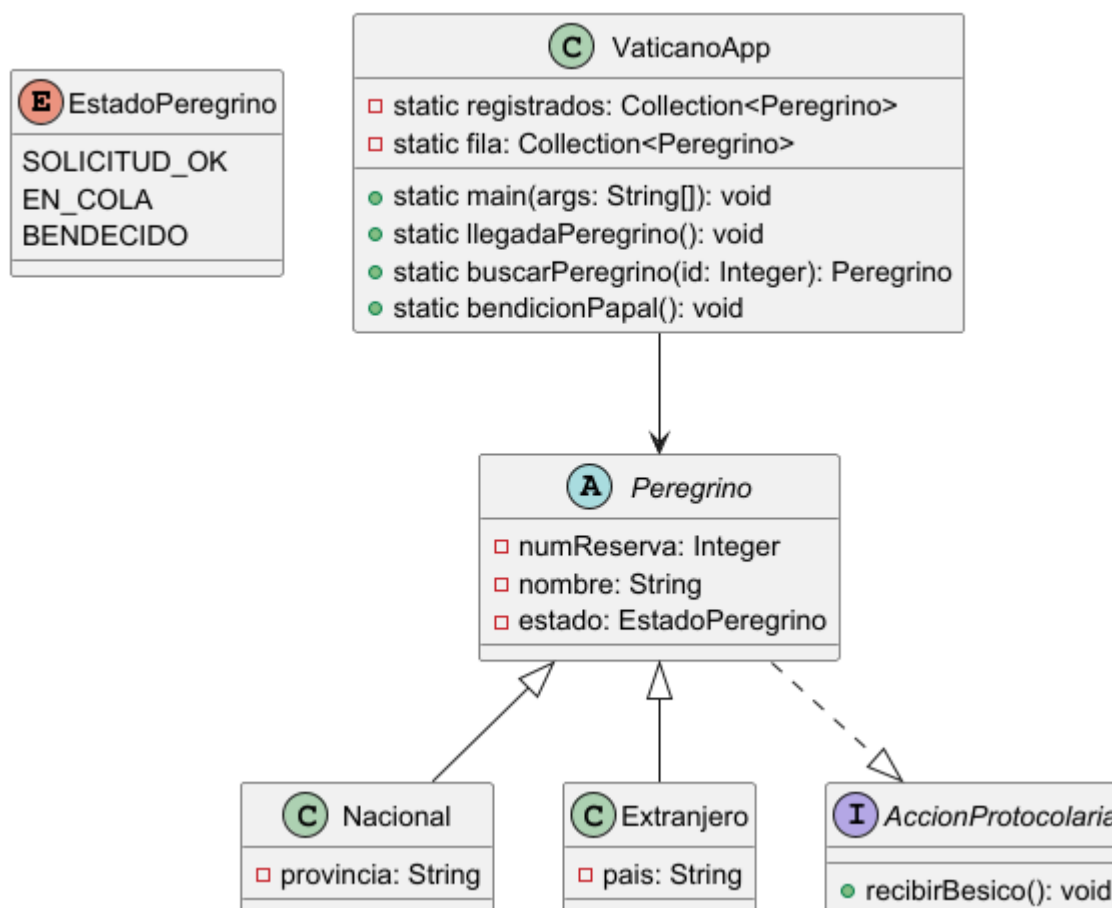
El programa deberá mostrar el porcentaje (%) que tienes tú de culpa:

```
Introduce el porcentaje de los 4 factores externos (compi-pista-pala-pelota):
54 12 25 8
=====
Tienes un 1% de culpa.
```

(7,5p) PROBLEMA 2: LA VISITA DEL PAPA

Ante la llegada del Papa a España, la Conferencia Episcopal necesita un software para gestionar a los fieles que han solicitado una bendición personal. El sistema debe asegurar que sólo entren en la fila quienes tienen reserva y que nadie que ya haya sido bendecido intente volver a ponerse en la fila para evitar aglomeraciones.

El diagrama que ha pensado el informático papal para la futura *app* es el siguiente:



Realiza un proyecto en *Java* para implementar la aplicación dada. Tienes libertad para modificar cualquier parte de la estructura, siempre que respetes la funcionalidad que se pide a continuación. Usa *Lombok* si te resulta útil. No se permite el uso de funciones *lambdas* y *streams*.

Condiciones para la construcción de las clases y otras cosas a tener en cuenta (LEE BIEN)

- (1,5p) Crea la estructura de clases correspondiente al diagrama UML proporcionado.
- (1,5p) Carga manual de datos: al arrancar, haz que el programa llene la lista de **registrados** con **5 peregrinos (3 nacionales y 2 extranjeros)** para tener datos de prueba.
 - Ten en cuenta que **no puede haber dos peregrinos con *numReserva* iguales.**
 - Muestra el contenido de la lista para comprobar que se han insertado correctamente.

c) **(2p) Registro en la fila (*llegadaPeregrino()*)**: el sistema pedirá por teclado el *numReserva*. A continuación, llamará al método *buscarPeregrino(Integer numReserva)* para buscar al peregrino en la lista de **registrados**.

- Si no existe, muestra: "**Error: Reserva no encontrada**" y devuelve **null**.

```
> Introduce num reserva: 500
[!] Error: Reserva no encontrada
```

- Si existe pero el atributo **estado** del peregrino ya es **BENDECIDO**, muestra: "**¡Pecador! Ya has recibido tu besico, deja paso a otros**" y devuelve **null**.

```
> Introduce num reserva: 101
[!] ¡Pecador! Ya has recibido tu besico, deja paso a otros.
```

- Si existe y no ha sido bendecido, se le cambia el estado a **EN_COLA** y devuelve el objeto **Peregrino** encontrado. En este caso, el método *llegadaPeregrino()* lo añade a la **fila** y muestra: "**Fiel [Nombre] añadido a la fila**".

```
> Introduce num reserva: 101
[+] Peregrino Juan añadido a la fila.
```

En caso de que lo que hayamos devuelto haya sido un **null**, este método no hará nada más.

d) **(2p) Bendición y despacho (*bendicionPapal()*)**: este método simula al Papa atendiendo a la fila. Ten en cuenta que debe despachar primero al peregrino que más tiempo lleva esperando (el primero que se ha puesto en la fila).

- Si la fila está vacía, muestra: "**Su Santidad está descansando, no hay nadie en la cola**".
- Si hay algún peregrino en la fila, el programa lanza el método *recibirBesico()* para cada uno y los echa ordenadamente de la fila. Este método imprimirá: "**El Papa le da un besico en la frente a [Nombre] y l@ despacha con una sonrisa**".

```
> El Papa bendice:
[!] El Papa le da un besico en la frente a Iván y l@ despacha con una sonrisa.
[!] El Papa le da un besico en la frente a Patri y l@ despacha con una sonrisa.
[!] El Papa le da un besico en la frente a Kevin y l@ despacha con una sonrisa.
```

(!) IMPORTANTE: se debe modificar el **estado** del *Peregrino* a **BENDECIDO** en la lista de registrados para que no pueda volver a entrar.

e) **(0,5p)** Muestra un listado de los **peregrinos extranjeros** que han sido bendecidos por el Papa.